

可以写，但我先强调一句：

Treap 做法理论更优，思路也很漂亮，但实现复杂度明显高于分块。正式比赛我仍然更推荐分块。

Treap 做法的核心是：

加一座桥，相当于把两条路径在高度 b 切开，然后交换上半段。

1. Treap 做法的核心模型

每个国家原本是一条竖线：

```
country a: 1 -> 2 -> 3 -> ... -> m + 1
```

初始时有 n 条路径：

```
路径 1：国家 1 的整条竖线
路径 2：国家 2 的整条竖线
...
路径 n：国家 n 的整条竖线
```

现在加入一座桥：

```
1 a b
```

连接：

```
(a, b) 和 (a + 1, b)
```

假设当前经过 (a, b) 的路径是 P ，经过 $(a + 1, b)$ 的路径是 Q 。

那么加桥之后，路径变成：

```
P 的下半段 + Q 的上半段
Q 的下半段 + P 的上半段
```

也就是交换两条路径的上半部分。

这非常适合用 **隐式 Treap** 维护每条路径的“竖线段序列”。

2. 路径怎么表示？

一条路径由若干段竖线组成。

例如：

国家 1 的 $[1, 3]$
国家 2 的 $[3, 7]$
国家 4 的 $[7, m + 1]$

表示从国家 1 高度 1 走到高度 3，然后通过桥到国家 2，高度 3 继续往上，之后再通过桥到国家 4。

每一段用一个 Treap 节点表示：

col = 属于哪个国家
 l, r = 这段竖线覆盖的高度区间

Treap 中节点顺序就是路径从下到上的顺序。

3. 加桥时做什么？

对于操作：

$1 \ a \ b$

我们需要找到：

当前包含 (a, b) 的竖线段 x
当前包含 $(a + 1, b)$ 的竖线段 y

然后分别把它们切开。

假设路径 P 是：

$P = P_down + P_up$

路径 Q 是：

$Q = Q_down + Q_up$

加入桥之后：

$newP = P_down + Q_up$
 $newQ = Q_down + P_up$

用 Treap 就是：

split path P at height b
split path Q at height b
merge(P_down , Q_up)
merge(Q_down , P_up)

查询 $2 \ a$ 时，只需要找到从国家 a 底部出发的那条 Treap，答案就是这棵 Treap 最右端竖线段的国家编号。

4. 怎么快速找到 (a, b) 所在竖线段?

对每个国家 a 维护一个 `set`，里面放当前属于这个国家的所有竖线段。

每个竖线段是区间 $[l, r]$ 。

查询 (a, b) 时，在 `set[a]` 中找 $l \leq b$ 的最后一个区间，检查它是否覆盖 b 。

由于题目保证新桥不会和已有桥共用端点，所以不会出现真正歧义。

5. 完整 C++23 代码

这份代码是 Treap 写法，复杂度大约：

```
O(q log(n + q))
#include <bits/stdc++.h>
using namespace std;

using i64 = long long;

struct Node {
    int ch[2]{};
    int p = 0;
    int sz = 1;
    int pri;
    int col, l, r;
    int last;
};

mt19937 rng(chrono::steady_clock::now().time_since_epoch().count());

int main() {
    cin.tie(nullptr)->sync_with_stdio(false);

    int n, m, q;
    cin >> n >> m >> q;

    vector<Node> tr(n + 2 * q + 5);
    vector<int> bot(n + 1);
    vector<set<pair<int, int>>> seg(n + 1);

    int tot = 0;

    auto isz = [&](int x) {
        return x ? tr[x].sz : 0;
    };

    auto setp = [&](int x, int p) {
        if (x) tr[x].p = p;
    };
```

```

auto pull = [&](int x) {
    tr[x].sz = 1 + isz(tr[x].ch[0]) + isz(tr[x].ch[1]);
    tr[x].last = tr[x].ch[1] ? tr[tr[x].ch[1]].last : tr[x].col;
    setp(tr[x].ch[0], x);
    setp(tr[x].ch[1], x);
};

auto newNode = [&](int col, int l, int r) {
    int x = ++tot;
    tr[x].ch[0] = tr[x].ch[1] = tr[x].p = 0;
    tr[x].sz = 1;
    tr[x].pri = int(rng());
    tr[x].col = col;
    tr[x].l = l;
    tr[x].r = r;
    tr[x].last = col;
    seg[col].insert({l, x});
    return x;
};

function<int(int)> root = [&](int x) {
    while (tr[x].p) x = tr[x].p;
    return x;
};

function<int(int)> rank = [&](int x) {
    int res = isz(tr[x].ch[0]) + 1;
    while (tr[x].p) {
        int y = tr[x].p;
        if (tr[y].ch[1] == x) {
            res += isz(tr[y].ch[0]) + 1;
        }
        x = y;
    }
    return res;
};

function<pair<int, int>(int, int)> split = [&](int x, int k) -> pair<int, int> {
    if (!x) return {0, 0};
    if (isz(tr[x].ch[0]) >= k) {
        auto [a, b] = split(tr[x].ch[0], k);
        tr[x].ch[0] = b;
        pull(x);
        setp(a, 0);
        setp(x, 0);
        return {a, x};
    } else {
        auto [a, b] = split(tr[x].ch[1], k - isz(tr[x].ch[0]) - 1);
        tr[x].ch[1] = a;
        pull(x);
        setp(x, 0);
        setp(b, 0);
    }
};

```

```

        return {x, b};
    }
};

function<int(int, int)> merge = [&](int x, int y) -> int {
    if (!x) {
        setp(y, 0);
        return y;
    }
    if (!y) {
        setp(x, 0);
        return x;
    }
    if (tr[x].pri < tr[y].pri) {
        tr[x].ch[1] = merge(tr[x].ch[1], y);
        pull(x);
        setp(x, 0);
        return x;
    } else {
        tr[y].ch[0] = merge(x, tr[y].ch[0]);
        pull(y);
        setp(y, 0);
        return y;
    }
};

auto findSeg = [&](int col, int h) {
    auto it = seg[col].upper_bound({h, INT_MAX});
    --it;
    return it->second;
};

auto cut = [&](int x, int h) {
    int rt = root(x);
    int rk = rank(x);

    seg[tr[x].col].erase({tr[x].l, x});

    int col = tr[x].col;
    int oldr = tr[x].r;

    tr[x].r = h;
    seg[col].insert({tr[x].l, x});

    int y = newNode(col, h, oldr);

    auto [a, b] = split(rt, rk);
    return pair<int, int>{a, merge(y, b)};
};

for (int i = 1; i <= n; i++) {
    bot[i] = newNode(i, 1, m + 1);
}

```

```

while (q--) {
    int op, a;
    cin >> op >> a;

    if (op == 1) {
        int b;
        cin >> b;

        int x = findSeg(a, b);
        int y = findSeg(a + 1, b);

        auto [x0, x1] = cut(x, b);
        auto [y0, y1] = cut(y, b);

        merge(x0, y1);
        merge(y0, x1);
    } else {
        int rt = root(bot[a]);
        cout << tr[rt].last << '\n';
    }
}

return 0;
}

```

6. 代码关键点解释

Node

```

struct Node {
    int ch[2]{};
    int p = 0;
    int sz = 1;
    int pri;
    int col, l, r;
    int last;
};

```

每个 Treap 节点代表一段竖线。

col

表示这段在哪个国家。

l, r

表示高度区间 $[l, r]$ 。

```
last
```

表示这棵 Treap 最右端，也就是路径最上端的国家编号。

查询时答案就是根节点的 `last`。

`bot[a]`

```
vector<int> bot(n + 1);
```

`bot[a]` 表示从国家 `a` 底部出发的路径中的一个固定节点。

初始时：

```
bot[a] = 国家 a 的整条竖线
```

之后即使路径被切开、合并，这个节点仍然留在从国家 `a` 底部出发的那条路径里。

所以查询 `2 a` 时：

```
int rt = root(bot[a]);  
cout << tr[rt].last << '\n';
```

`seg[col]`

```
vector<set<pair<int, int>>> seg(n + 1);
```

`seg[col]` 存国家 `col` 当前有哪些竖线段。

每个元素是：

```
{l, node_id}
```

找 `(col, h)` 所在段时：

```
auto it = seg[col].upper_bound({h, INT_MAX});  
--it;  
return it->second;
```

也就是找起点 `l <= h` 的最后一段。

`cut(x, h)`

这是最重要的函数。

它把节点 `x` 所在的路径，在高度 `h` 处切成上下两部分。

原本节点 `x` 是：

```
[col, l, r]
```

切完后变成两段：

```
[col, l, h]  
[col, h, r]
```

第一段留在原节点 `x`，第二段新建节点 `y`。

然后把整条 Treap 在 `x` 后面切开：

```
auto [a, b] = split(rt, rk);
```

其中：

```
a = 下半段路径  
b = 原来 x 后面的路径
```

最后：

```
上半段 = merge(y, b)
```

所以返回：

```
{下半段, 上半段}
```

加桥

```
int x = findSeg(a, b);  
int y = findSeg(a + 1, b);  
  
auto [x0, x1] = cut(x, b);  
auto [y0, y1] = cut(y, b);  
  
merge(x0, y1);  
merge(y0, x1);
```

含义非常直观：

```
x0 = 第一条路径下半段  
x1 = 第一条路径上半段  
  
y0 = 第二条路径下半段  
y1 = 第二条路径上半段
```


加桥之后交换上半段：

```
x0 + y1  
y0 + x1
```

7. 这个写法的坑

这个 Treap 做法比分块难很多，主要坑在这些地方：

1. 必须维护父指针

因为查询需要：

```
root(bot[a])
```

加桥时也需要知道某个节点在它所在 Treap 里的排名：

```
rank(x)
```

所以 Treap 节点必须有 `p`。

2. `split` 和 `merge` 后要维护父亲

每次修改儿子后都要 `pull(x)`。

`pull(x)` 里不仅更新 `sz` 和 `last`，还要更新儿子的父亲。

3. 每次切竖线段时，要同步更新 `seg`

原节点 `[l, r]` 被改成 `[l, h]`，还新建 `[h, r]`。

所以必须：

```
erase old interval  
insert lower interval  
insert upper interval
```

否则后面找 `(a, b)` 所在段会错。

4. 区间端点会重合

切 `[l, r]` 于 `h` 后变成：

```
[l, h]  
[h, r]
```

两个区间都包含高度 h 。

这在数学上看起来有点怪，但题目保证以后不会再在同一个节点加桥，所以不会影响正确性。

8. 和分块相比

Treap 做法：

时间复杂度更优
代码更难
边界更多
调试成本高

分块做法：

复杂度略差
代码短很多
思路更稳
比赛更推荐

所以我的建议是：

如果你是为了这题 AC，用分块。

如果你想练高级动态路径维护，可以研究这份 Treap。